

for products that are developed from scratch, and for new systems. Leaders have the luxury of a good TTM, whereas followers are always looking for more time. In other words, for a 'blue sky innovation' product, or a 'new paradigm' product, TTM is not a consideration.

Challenges in platform choice

Before choosing an operating system for a given requirement, architects should explore all possible details about the operating system. We now delve into the many questions that the architect should ask, under various areas.

Kernel features

- What are the processor architectures the kernel needs to support (such as multi-core/multi-processor support)?
- What is the type of kernel required (monolithic, micro or hybrid kernel)?
- What is the footprint (size) of the kernel?
- Can I configure the kernel according to my requirements, and reduce the size of the kernel?
- Will the kernel support dynamic kernel module loading?
- How do I efficiently allocate resources for my kernel modules?
 - Resources can be I/O port addresses, Interrupt Request Lines (IRQs), memory space allocation, etc.
- Will the kernel support sharing IRQs for many devices?
- What kernel mechanisms exist to reduce interrupt latency?
- How can I write a very efficient Interrupt Service Routine (ISR) for a given scenario, deferring work in the ISR to minimise interrupt disable time?
- How do I include custom features in the kernel, apart from the standard set of services?
- How can I protect privileged kernel features from accidental or intentional access by a normal user?
- How do I share common functions between different kernel modules?
- What are the different ways of transferring data between kernel-space and user-space?
- How do I synchronise a critical section that may be shared by a process context or interrupt context?
- What are the available kernel locking mechanisms? (Fine-grained locking mechanisms are needed to protect a shared single integer variable, or a hardware bit.)
- Is there any lock-avoidance mechanism available, to protect a critical section from a race condition?
- How do I handle errors in a kernel module?
- How do I identify a deadlock situation in a kernel module?
- How do I do event-driven programming in a kernel module?
- What devices does the kernel support (available device drivers)?
- How do I identify unwanted code in the kernel source?
- How do I forcefully unmount my devices?
- What are the kernel crash scenarios?
- How do I protect critical data in RAM from kernel crashes?

- How do I reboot the system quickly, bypassing BIOS/firmware?
- What kernel debugging and tracing tools are available?

File management

- What are the file interfaces available?
- How can we modify the existing file system size without affecting the stored data?
- What are the file systems the kernel supports?
- What are the embedded file systems the kernel will support?
- What is the maximum size limit for each file system?
- What are the available pseudo file systems and their usage?
- What file-locking synchronisation mechanisms does the kernel provide?
- What limitations does the OS have in areas such as file size, number of open files within a process, back up facility, etc?

Timers

What are the different timers available in the kernel (see the list below), and what is our choice, based on application and product requirements?

- Monotonic timers
- Periodic timers
- High-resolution timers
- Real-time timers
- POSIX timers
- Hardware timers

Memory management

Our choice of a memory-management policy depends on speed and space requirements, and on whether the application is running on a server, an embedded system or a desktop. Some of the memory management and protection mechanisms available include:

- Virtual memory support
- Kernel space protection
- User-space protection from other processes
- Shared memory support
- Disk traffic time reduction (buffer cache, sync())
- Minimise fragmentation (thrashing, page allocation, internal fragmentation, etc)

We also need to know:

- How to extend swap size dynamically
- How to avoid swapping, and lock the process space in physical memory

Process/thread management

We need to understand the following process and thread subsystem parameters before we start to use the system:

- Usage of processes and threads
- Timer interrupt configuration
- Choice of scheduling policy, and the method to set it
- Available priority range (dynamic versus static)
- Pre-emptive versus non-pre-emptive, and whether it's configurable
- Support for real-time process and threads
- Support for signals